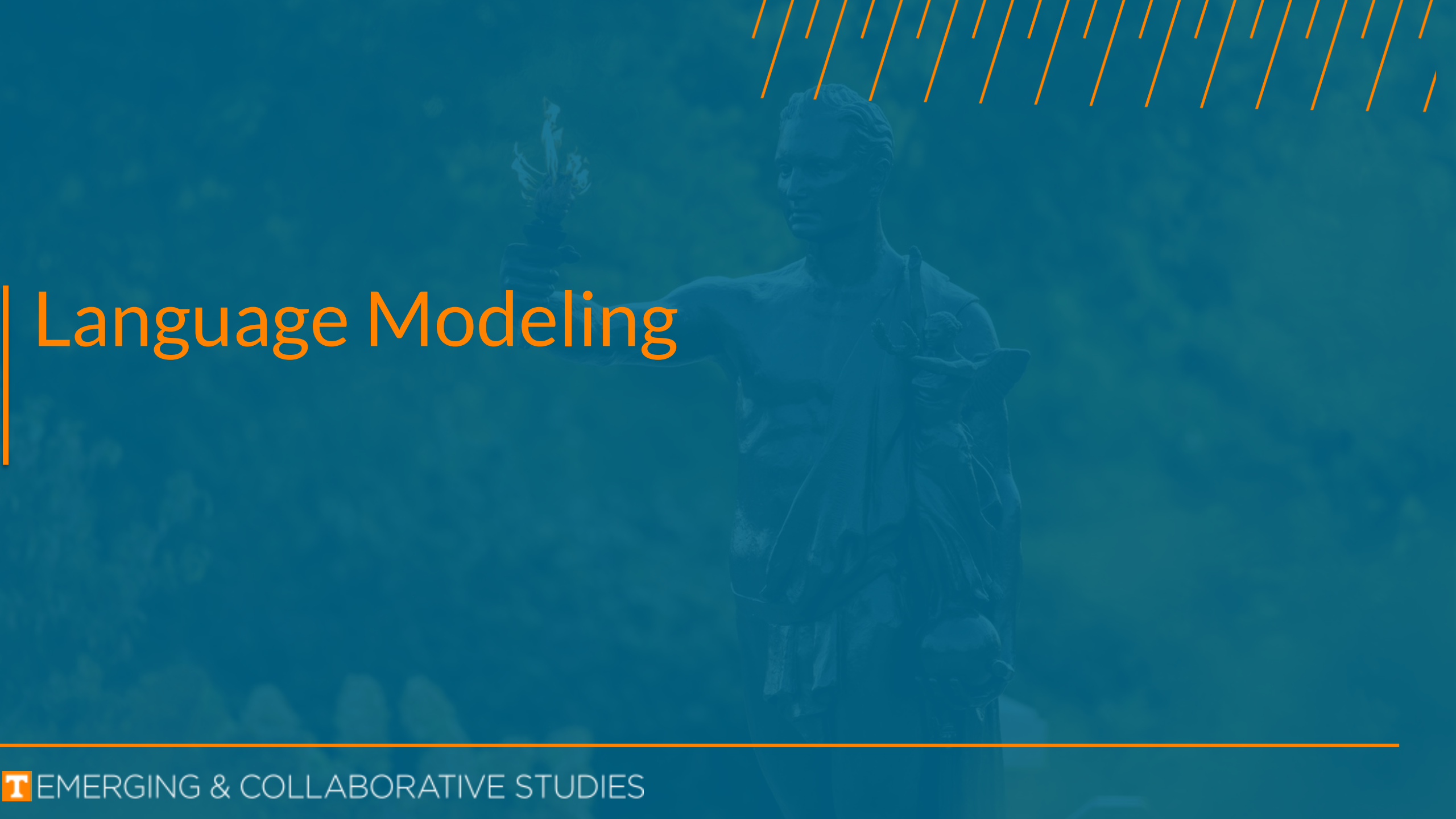

AI-102 Natural Language Processing (NLP)

Dr. Jimmy Ardiansyah



EMERGING &
COLLABORATIVE STUDIES





Language Modeling

What's N-gram?

N-gram language models are a simple but effective way to do this using a **statistical approach** based on counting word sequences in a large body of text, called a **corpus**.

The probability of a sequence of words is approximately equal to the product of the probability of each word given the few words before it.

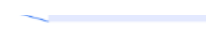
What's N-gram?

N-gram language models are a simple but effective way to do this using a **statistical approach** based on counting word sequences in a large body of text, called a **corpus**.

The probability of a sequence of words is approximately equal to the product of the probability of each word given the few words before it.

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i | w_{i-(N-1)}, \dots, w_{i-1})$$

- $P(w_1, w_2, \dots, w_n)$ → "the probability of the sequence of words w_1 through w_n "
- $\approx$ → "is approximately equal to"
- $\prod$ → "the product of" or "multiply together"
- $P(w_i | w_{i-(N-1)}, \dots, w_{i-1})$ → "the probability of word w_i given the previous N minus one words"



What's N-gram?

N-gram language models are a simple but effective way to do this using a **statistical approach** based on counting word sequences in a large body of text, called a **corpus**.

The probability of a sequence of words is approximately equal to the product of the probability of each word given the few words before it.

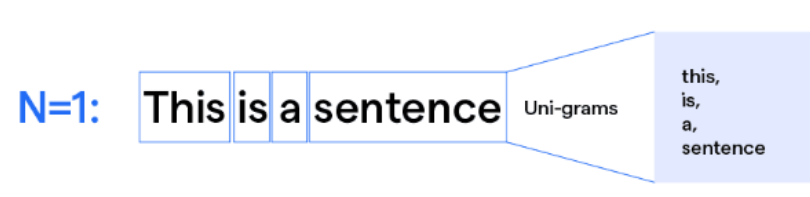
$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i | w_{i-(N-1)}, \dots, w_{i-1})$$

- $P(w_1, w_2, \dots, w_n) \rightarrow$ "the probability of the sequence of words w one through w_n "
- $\approx \rightarrow$ "is approximately equal to"
- $\prod \rightarrow$ "the product of" or "multiply together"
- $P(w_i | w_{i-(N-1)}, \dots, w_{i-1}) \rightarrow$ "the probability of word w_i given the previous N minus one words"

Type of N-gram

- Unigram (N=1)

A unigram model is the simplest, and it doesn't look at any previous words. It just predicts the next word based on how frequently each word appears in the corpus.



What's N-gram?

N-gram language models are a simple but effective way to do this using a **statistical approach** based on counting word sequences in a large body of text, called a **corpus**.

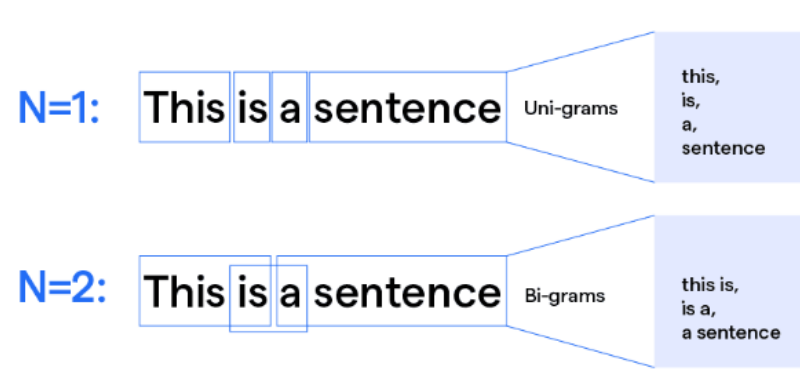
The probability of a sequence of words is approximately equal to the product of the probability of each word given the few words before it.

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i | w_{i-(N-1)}, \dots, w_{i-1})$$

- $P(w_1, w_2, \dots, w_n) \rightarrow$ "the probability of the sequence of words w one through w_n "
- $\approx \rightarrow$ "is approximately equal to"
- $\prod \rightarrow$ "the product of" or "multiply together"
- $P(w_i | w_{i-(N-1)}, \dots, w_{i-1}) \rightarrow$ "the probability of word w_i given the previous N minus one words"

Type of N-gram

- **Unigram (N=1)**
A unigram model is the simplest, and it doesn't look at any previous words. It just predicts the next word based on how frequently each word appears in the corpus.
- **Bigram (N=2)**
A bigram model looks at the **one** previous word to predict the next. This is the most common and easy-to-understand N-gram model



What's N-gram?

N-gram language models are a simple but effective way to do this using a **statistical approach** based on counting word sequences in a large body of text, called a **corpus**.

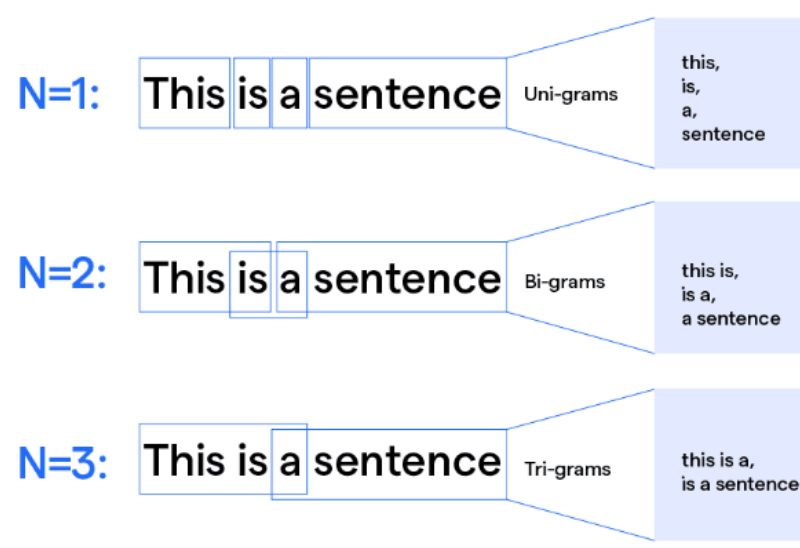
The probability of a sequence of words is approximately equal to the product of the probability of each word given the few words before it.

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i | w_{i-(N-1)}, \dots, w_{i-1})$$

- $P(w_1, w_2, \dots, w_n) \rightarrow$ "the probability of the sequence of words w one through w n "
- $\approx \rightarrow$ "is approximately equal to"
- $\prod \rightarrow$ "the product of" or "multiply together"
- $P(w_i | w_{i-(N-1)}, \dots, w_{i-1}) \rightarrow$ "the probability of word w i given the previous N minus one words"

Type of N-gram

- **Unigram (N=1)**
A unigram model is the simplest, and it doesn't look at any previous words. It just predicts the next word based on how frequently each word appears in the corpus.
- **Bigram (N=2)**
A bigram model looks at the **one** previous word to predict the next. This is the most common and easy-to-understand N-gram model
- **Trigram (N=3)**
A trigram model considers the **two** previous words. This gives it more context and often leads to better predictions.



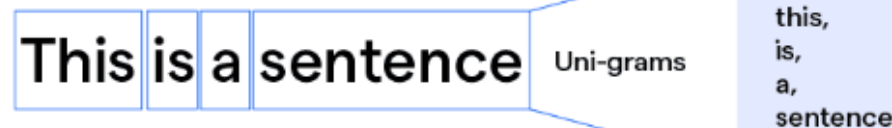
Generating N-Gram in Python

```
from nltk import ngrams
import nltk
nltk.download('punkt')

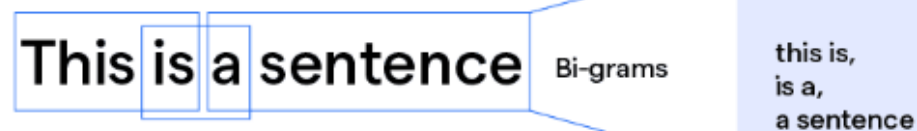
text = "This is a sentence."
tokens = nltk.word_tokenize(text)

def generate_ngrams(tokens, n):
    return [' '.join(gram) for gram in ngrams(tokens, n)]
```

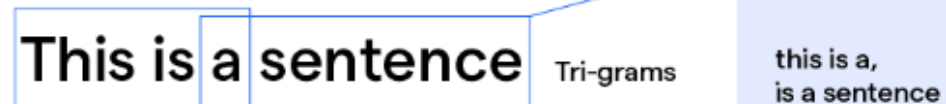
N=1:



N=2:



N=3:



Challenges of N-gram Models

- **Sparsity:** Huge number of possible N-grams; many never seen in training data.
- **Limited Context:** Only looks at N-1 previous words; misses long-range dependencies.
- **Memory Usage:** Large N $\rightarrow$ huge storage for probabilities.
- **Poor Generalization:** Fails on unseen word combinations.
- **OOV Words:** Can't handle new words not in training data.
- **No Semantics:** Counts word patterns but doesn't understand meaning.



Applications of N-gram Models

- **Text Prediction:** autocomplete & keyboard suggestions.
- **Speech Recognition:** choose most probable word sequence.
- **Machine Translation:** contextual accuracy across languages.
- **Text Generation:** create coherent sentences (chatbots, story generation).



"I hate to be a bother, young man, but do you speak emoji?
My grandson said '👁👁👁' - should I call 911?"

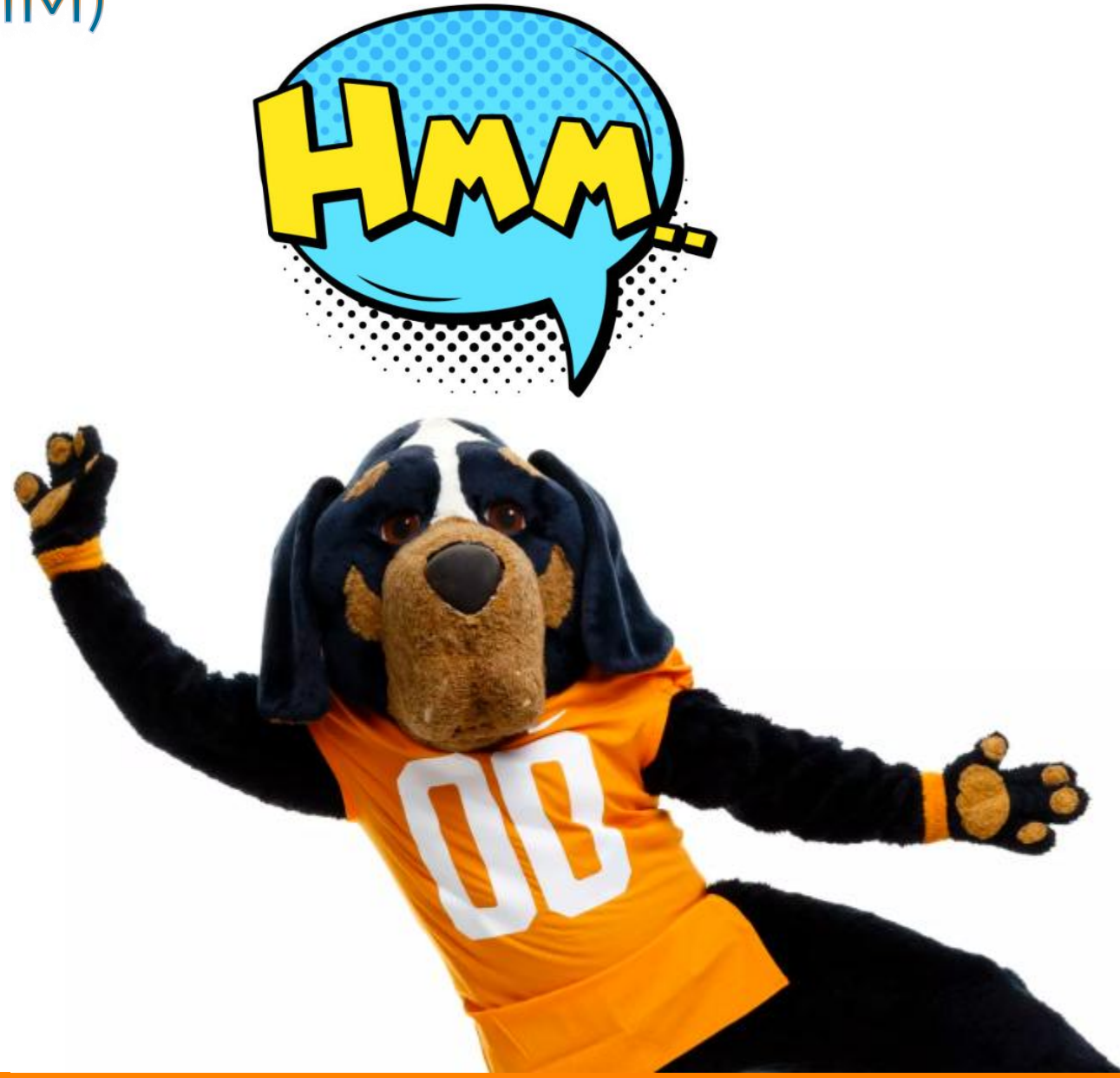
Hidden Markov Models (HMM)

A *Hidden Markov Model (HMM)* is a probabilistic model for analyzing **sequences** where the actual (hidden) states are not directly observed.

Core Idea: Observed events (like words) are generated by hidden states (like part-of-speech tags) and capture the relationships between **visible sequences** and **hidden structures** in data.

Common Applications:

- Part-of-Speech (PoS)
- Named Entity Recognition (NER)
- Speech Recognition



Hidden Markov Models (HMM)

A Hidden Markov Model (HMM) is a statistical model that helps us understand a sequence of events where we can see the output, but not the process that created it. It's called "hidden" because the key states that drive the process are **unobservable**

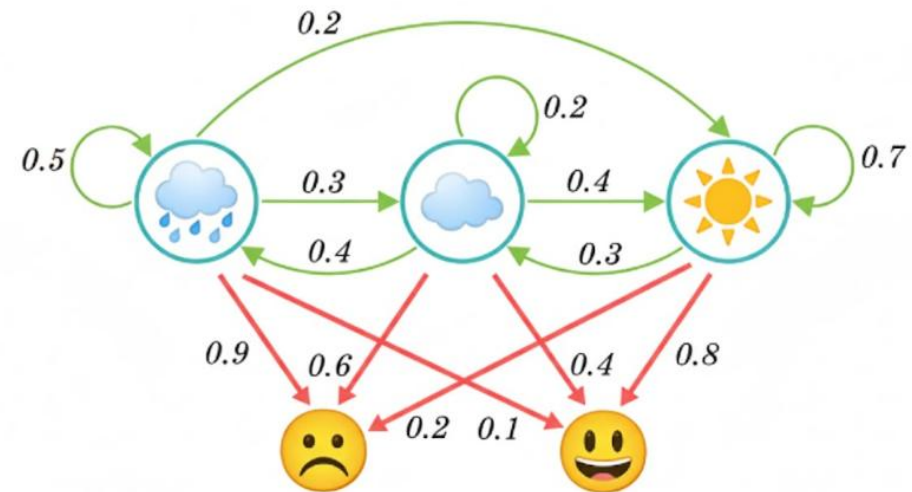
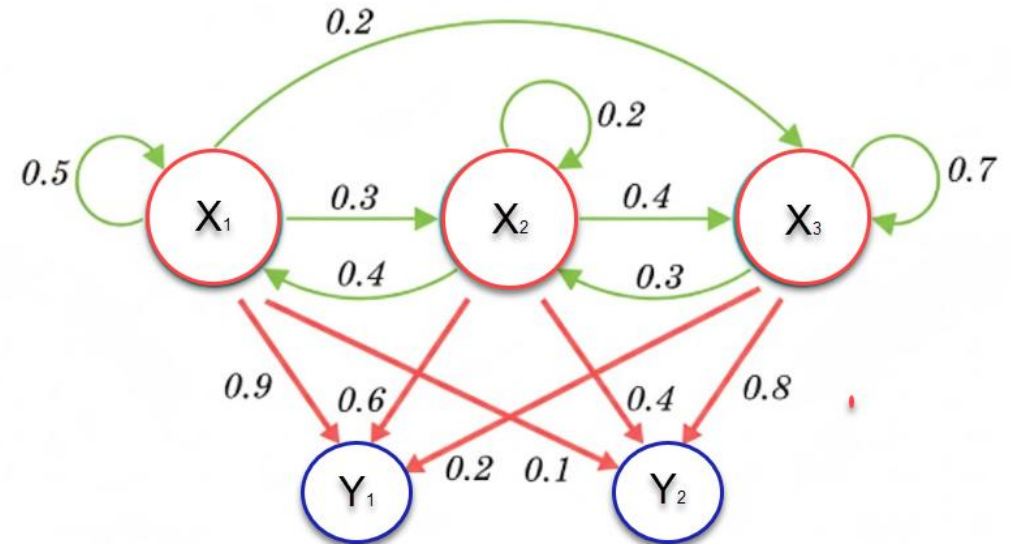
An HMM is built on these key components:

States (X_n): The hidden, unobservable variables (e.g., grammatical categories like "noun" or "verb") that generate the visible data.

Observations (Y_n): The visible data points (e.g., the words in a sentence) that are generated by the hidden states.

Probabilities:

- **Transition Probabilities:** These describe the probability of moving from one **hidden state** to another.
- **Emission Probabilities:** These describe the probability of an **observed event** being produced from a specific **hidden state**.



Three Fundamental Problem of HMM

Evaluation:

Compute $P(\text{observation sequence} \mid \text{model}) \rightarrow$ How likely is the sequence?

Solved by: **Forward Algorithm**

Decoding:

Find the most probable hidden state sequence.

Solved by: **Viterbi Algorithm**

Learning:

Estimate model parameters (transition & emission probabilities) from data.

Solved by: **Baum-Welch (EM) Algorithm**



Recurrent Neural Networks (RNNs)



Recurrent Neural Networks (RNNs) are a type of neural network designed to handle **sequential data**, like words in a sentence. Unlike a traditional neural network that processes each input independently, an RNN has a "memory" that allows it to remember information from previous inputs. RNNs are often characterized by"

Key Idea:

RNNs have **loops** that allow information to persist, giving them a kind of "memory" of previous inputs.

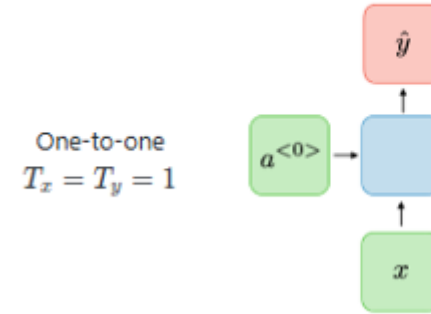
Common Applications:

- Text generation & sentiment analysis
- Machine translation
- Speech recognition
- Stock or time-series forecasting

RNN Input-Output Types

One-to-One

This RNN type channels one input to one outputs. It enables linguistic applications like image captioning by generating a sentence from a single keyword.

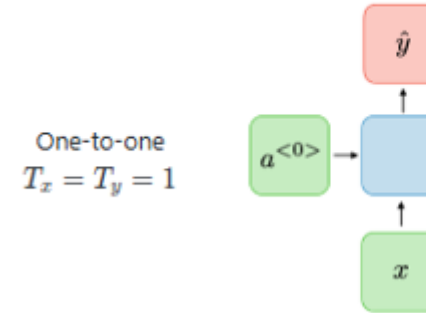


● (Input), ● (Hidden Layer), ● (Output Layer)

RNN Input-Output Types

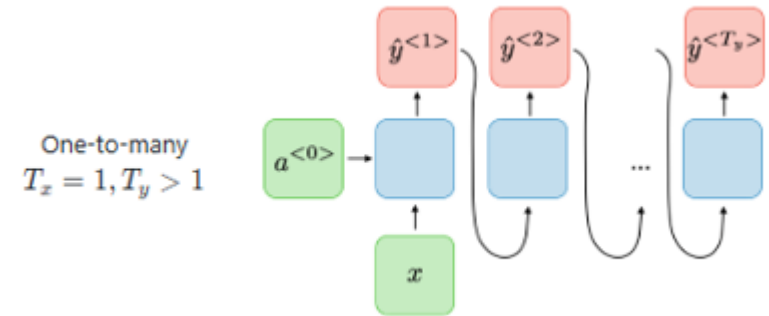
One-to-One

This RNN type channels one input to one outputs. It enables linguistic applications like image captioning by generating a sentence from a single keyword.



One-to-Many

The model uses one input to predict multiple outputs. For example, you can create a language translator with an RNN, which analyzes a sentence and correctly structures the words in a different language.

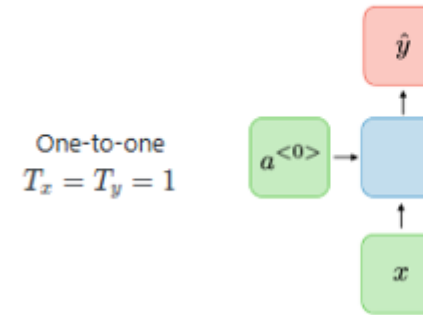


● (Input), ● (Hidden Layer), ● (Output Layer)

RNN Input-Output Types

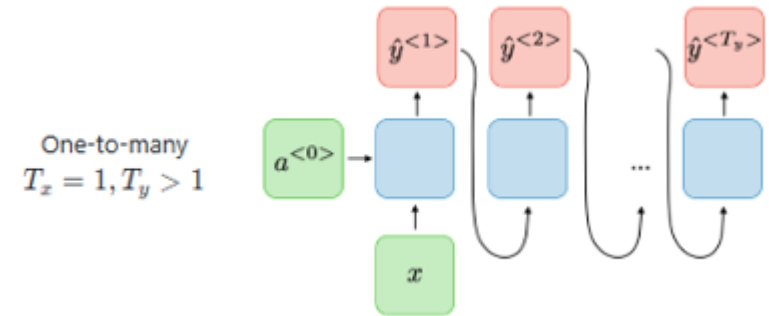
One-to-One

This RNN type channels one input to one outputs. It enables linguistic applications like image captioning by generating a sentence from a single keyword.



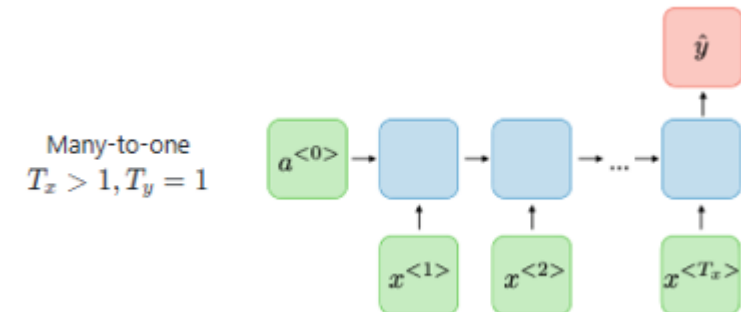
One-to-Many

The model uses one input to predict multiple outputs. For example, you can create a language translator with an RNN, which analyzes a sentence and correctly structures the words in a different language.



Many-to-One

Several inputs are mapped to an output. This is helpful in applications like sentiment analysis, where the model predicts customers' sentiments like *positive*, *negative*, and *neutral* from input testimonials.



● (Input), ● (Hidden Layer), ● (Output Layer)

Challenges with RNN

I thought RNNs would be perfect language models. Turns out they have the memory span of a goldfish.



Vanishing Gradients – gradients shrink over time; network forgets long-term context.

Exploding Gradients – gradients grow uncontrollably, causing unstable training.

Long-Term Dependencies – struggles to retain information over many time steps.

Computational Cost – sequential processing slows training and limits parallelization.

Overfitting – can memorize training data if not regularized.

Difficult to Tune – sensitive to hyperparameters and initialization

Long Short-Term Memory (LSTM)



Basic RNNs struggle with long-term dependencies because of the vanishing gradient problem. As the network processes a long sequence, the information from the early steps gets "*washed out*," making it hard to remember and use that information later. **Long Short-Term Memory (LSTM)** networks are a special type of RNN designed specifically to overcome this problem. They are engineered to remember important information for longer periods.

|

.

Long Short-Term Memory (LSTM)

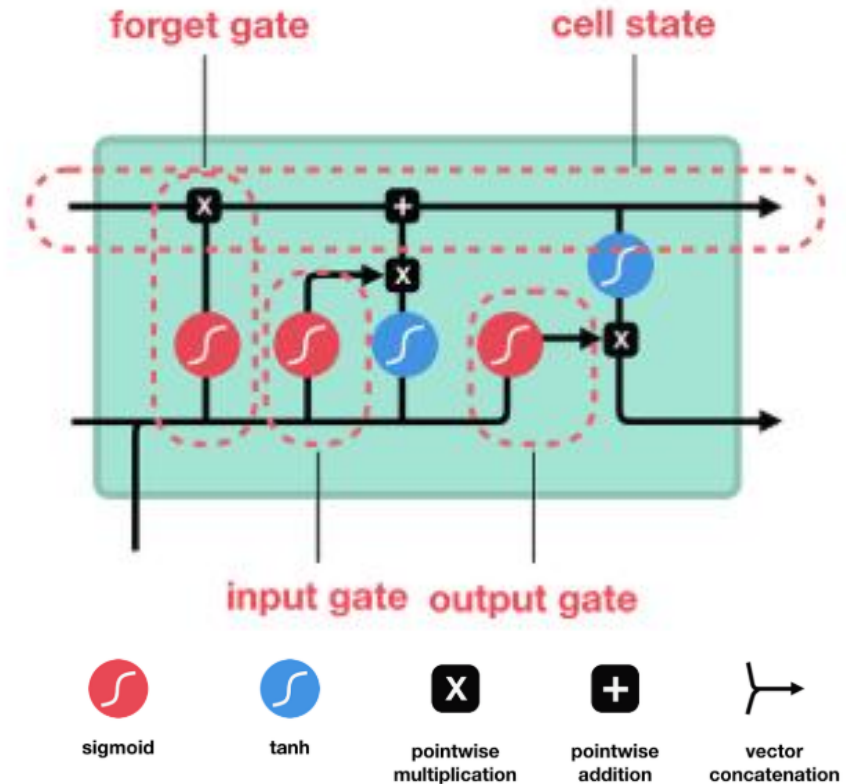
Basic RNNs struggle with long-term dependencies because of the vanishing gradient problem. As the network processes a long sequence, the information from the early steps gets "washed out," making it hard to remember and use that information later. **Long Short-Term Memory (LSTM)** networks are a special type of RNN designed specifically to overcome this problem. They are engineered to **remember important information for longer periods**.

How do LSTM work?

LSTMs add a persistent memory mechanism and gates that control information flow:

- They maintain a "memory cell" that preserves information over time, allowing information to persist across many steps.
- Gates learn to selectively retain or discard information..
- Alongside this long-term memory, the network also keeps a hidden state that functions like short-term memory for immediate computations

Put simply: the memory cell carries long-range context; the gates open or close to write to it, keep it, or read from it, enabling robust long-term dependency learning.



sigmoid function, which outputs a number between 0 and 1, to decide how much of something to let through. A 0 means "block completely," and a 1 means "let it all through"

Tanh function is tanh is a mathematical tool that helps the LSTM model regulate and normalize the flow of information, making the training process more stable and efficient.

Gated Recurrent Unit (GRU)

A **Gated Recurrent Unit (GRU)** is a type of Recurrent Neural Network (RNN) architecture designed to fix the vanishing gradient problem in traditional RNNs. It helps the model remember important information for longer and forget irrelevant details during training.

Gated Recurrent Unit (GRU)

A **Gated Recurrent Unit (GRU)** is a type of Recurrent Neural Network (RNN) architecture designed to fix the vanishing gradient problem in traditional RNNs. It helps the model remember important information for longer and forget irrelevant details during training.

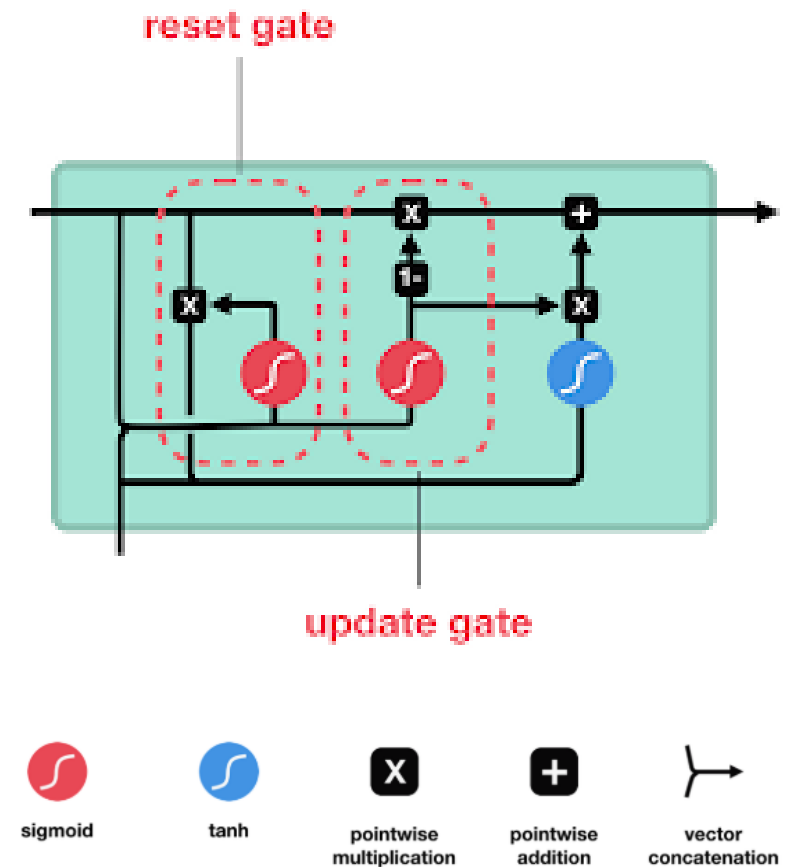
How do GRU work?

Like an RNN, a GRU processes sequential data (like sentences) one step at a time. But instead of a simple hidden state, it has **two gates** that control information flow:

They maintain a “memory cell” that preserves information over time, allowing information to persist across many steps.

- **Update Gate** — decides how much of the past information to keep.
- **Reset Gate** — decides how much of the previous memory to forget.

These gates work together to balance short-term and long-term memory efficiently.



Sigmoid function, which outputs a number between 0 and 1, to decide how much of something to let through. A 0 means "block completely," and a 1 means "let it all through"

Tanh function is tanh is a mathematical tool that helps the LSTM model regulate and normalize the flow of information, making the training process more stable and efficient.





EMERGING &
COLLABORATIVE
STUDIES
